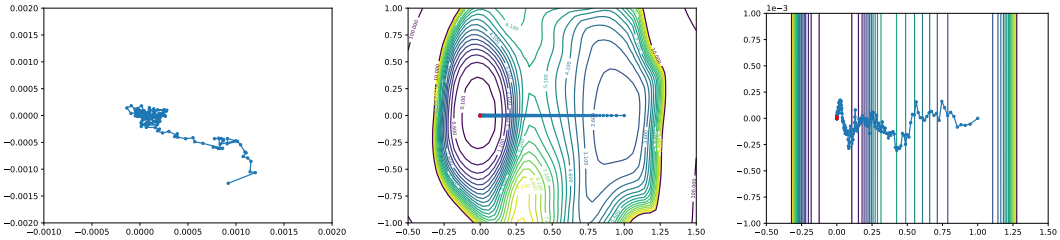


(a) Two random directions (b) Random direction for y-axis (c) Enlarged version (b)

Figure 8: Ineffective visualizations of optimizer trajectories. These visualizations suffer from the orthogonality of random directions in high dimensions.

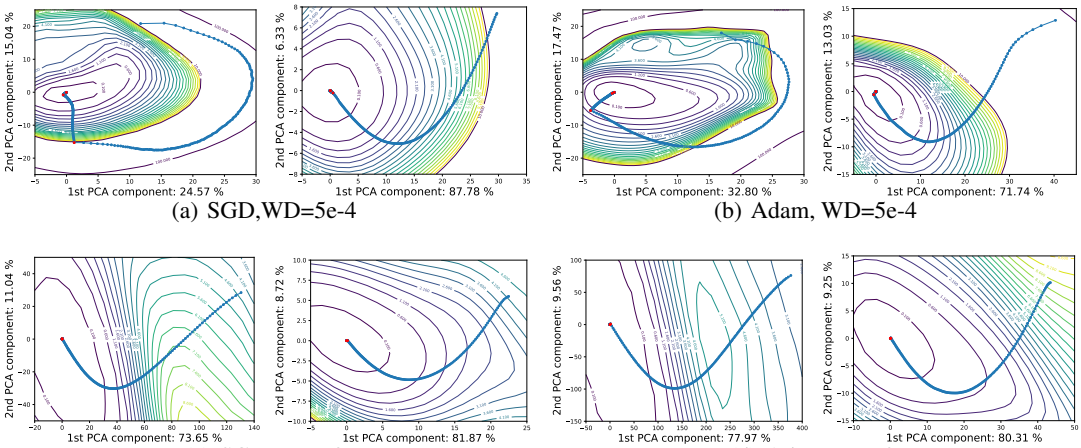
This is problematic when optimization trajectories lie in extremely low dimensional spaces. In this case, a randomly chosen vector will lie orthogonal to the low-rank space containing the optimization path, and a projection onto a random direction will capture almost no variation. Figure 8(b) suggests that optimization trajectories are low-dimensional because the random direction captures orders of magnitude less variation than the vector that points along the optimization path. Below, we use PCA directions to directly validate this low dimensionality, and also to produce effective visualizations.

7.2 Effective Trajectory Plotting using PCA Directions

To capture variation in trajectories, we need to use non-random (and carefully chosen) directions. Here, we suggest an approach based on PCA that allows us to measure how much variation we've captured; we also provide plots of these trajectories along the contours of the loss surface.

Let θ_i denote model parameters at epoch i, and the final parameters after n epochs of training are denoted θ_n. Given n training epochs, we can apply PCA to the matrix $M = [\theta_0 - \theta_n; \cdots; \theta_{n-1} - \theta_n]$, and then select the two most explanatory directions. Optimizer trajectories (blue dots) and loss surfaces along PCA directions are shown in Figure 9. Epochs where the learning rate was decreased are shown as red dots. On each axis, we measure the amount of variation in the descent path captured by that PCA direction.

At early stages of training, the paths tend to move perpendicular to the contours of the loss surface, i.e., along the gradient directions as one would expect from non-stochastic gradient descent. The stochasticity becomes fairly pronounced in several plots during the later stages of training. This is particularly true of the plots that use weight decay and small batches (which leads to more gradient



(a) SGD,WD=5e-4 (b) Adam, WD=5e-4

(c) SGD, WD=0 (d) Adam,WD=0

Figure 9: Projected learning trajectories use normalized PCA directions for VGG-9. The left plot in each subfigure uses batch size 128, and the right one uses batch size 8192.

noise, and a more radical departure from deterministic gradient directions). When weight decay and small batches are used, we see the path turn nearly parallel to the contours and "orbit" the solution when the stepsize is large. When the stepsize is dropped (at the red dot), the effective noise in the system decreases, and we see a kink in the path as the trajectory falls into the nearest local minimizer.

Finally, we can directly observe that the descent path is very low dimensional: between 40% and 90% of the variation in the descent paths lies in a space of only 2 dimensions. The optimization trajectories in Figure 9 appear to be dominated by movement in the direction of a nearby attractor. This low dimensionality is compatible with the observations in Section 6, where we observed that non-chaotic landscapes are dominated by wide, nearly convex minimizers.

8 Conclusion

We presented a visualization technique that provides insights into the consequences of a variety of choices facing the neural network practitioner, including network architecture, optimizer selection, and batch size. Neural networks have advanced dramatically in recent years, largely on the back of anecdotal knowledge and theoretical results with complex assumptions. For progress to continue to be made, a more general understanding of the structure of neural networks is needed. Our hope is that effective visualization, when coupled with continued advances in theory, can result in faster training, simpler models, and better generalization.

Acknowledgements

Li, Xu, and Goldstein were supported by the Office of Naval Research (N00014-17-1-2078), DARPA Lifelong Learning Machines (FA8650-18-2-7833), the DARPA YFA program (D18AP00055), and the Sloan Foundation. Taylor was supported by the ONR (N0001418WX01582), and the DOD HPC Modernization Program. Studer was supported in part by Xilinx, Inc. and by the US National Science Foundation (NSF) under grants ECCS-1408006, CCF-1535897, CCF-1652065, CNS-1717559, and ECCS-1824379.

References

[1] David Balduzzi, Marcus Frean, Lennox Leary, JP Lewis, Kurt Wan-Duo Ma, and Brian McWilliams. The shattered gradients problem: If resnets are the answer, then what is the question? In *ICML*, 2017.

[2] Avrim Blum and Ronald L Rivest. Training a 3-node neural network is np-complete. In *NIPS*, 1989.

[3] Pratik Chaudhari, Anna Choromanska, Stefano Soatto, and Yann LeCun. Entropy-sgd: Biasing gradient descent into wide valleys. In *ICLR*, 2017.

[4] Anna Choromanska, Mikael Henaff, Michael Mathieu, Gérard Ben Arous, and Yann LeCun. The loss surfaces of multilayer networks. In *AISTATS*, 2015.

[5] Yann N Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *NIPS*, 2014.

[6] Soham De, Abhay Yadav, David Jacobs, and Tom Goldstein. Automated inference with adaptive batches. In *AISTATS*, 2017.

[7] Laurent Dinh, Razvan Pascanu, Samy Bengio, and Yoshua Bengio. Sharp minima can generalize for deep nets. In *ICML*, 2017.

[8] Gintare Karolina Dziugaite and Daniel M Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. In *UAI*, 2017.

[9] C Daniel Freeman and Joan Bruna. Topology and geometry of half-rectified network optimization. In *ICLR*, 2017.

[10] Marcus Gallagher and Tom Downs. Visualization of learning in multilayer perceptron networks using principal component analysis. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 33(1):28–34, 2003.

[11] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, 2010.

[12] Tom Goldstein and Christoph Studer. Phasemax: Convex phase retrieval via basis pursuit. *arXiv preprint arXiv:1610.07531*, 2016.

[13] Ian J Goodfellow, Oriol Vinyals, and Andrew M Saxe. Qualitatively characterizing neural network optimization problems. In *ICLR*, 2015.

[14] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.

[15] Benjamin D Haeffele and René Vidal. Global optimality in neural network training. In *CVPR*, 2017.

[16] Moritz Hardt and Tengyu Ma. Identity matters in deep learning. In *ICLR*, 2017.

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *CVPR*, 2016.

[18] Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural Computation*, 9(1):1–42, 1997.

[19] Elad Hoffer, Itay Hubara, and Daniel Soudry. Train longer, generalize better: closing the generalization gap in large batch training of neural networks. *NIPS*, 2017.

[20] Gao Huang, Zhuang Liu, Kilian Q Weinberger, and Laurens van der Maaten. Densely connected convolutional networks. In *CVPR*, 2017.

[21] Daniel Jiwoong Im, Michael Tao, and Kristin Branson. An empirical analysis of deep network loss surfaces. *arXiv preprint arXiv:1612.04010*, 2016.

[22] Sergey Ioffe and Christian Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *ICML*, 2015.

[23] Kenji Kawaguchi, Leslie Pack Kaelbling, and Yoshua Bengio. Generalization in deep learning. *arXiv preprint arXiv:1710.05468*, 2017.

[24] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. In *ICLR*, 2017.

[25] Anders Krogh and John A Hertz. A simple weight decay can improve generalization. In *NIPS*, 1992.

[26] Yuanzhi Li and Yang Yuan. Convergence analysis of two-layer neural networks with relu activation. *arXiv preprint arXiv:1705.09886*, 2017.

[27] Qianli Liao and Tomaso Poggio. Theory of deep learning ii: Landscape of the empirical risk in deep learning. *arXiv preprint arXiv:1703.09833*, 2017.

[28] Zachary C Lipton. Stuck in a what? adventures in weight space. In *ICLR Workshop*, 2016.

[29] Eliana Lorch. Visualizing deep network training trajectories with pca. In *ICML Workshop on Visualization for Deep Learning*, 2016.

[30] Behnam Neyshabur, Srinadh Bhojanapalli, David McAllester, and Nati Srebro. Exploring generalization in deep learning. In *NIPS*, 2017.

[31] Quynh Nguyen and Matthias Hein. The loss surface of deep and wide neural networks. In *ICML*, 2017.

[32] Itay Safran and Ohad Shamir. On the quality of the initial basin in overspecified neural networks. In *ICML*, 2016.

[33] Karen Simonyan and Andrew Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *ICLR*, 2015.

[34] Leslie N Smith and Nicholay Topin. Exploring loss function topology with cyclical learning rates. *arXiv preprint arXiv:1702.04283*, 2017.

[35] Mahdi Soltanolkotabi, Adel Javanmard, and Jason D Lee. Theoretical insights into the optimization landscape of over-parameterized shallow neural networks. *arXiv preprint arXiv:1707.04926*, 2017.